Attention Is All You Need

注意力机制就是你需要的一切

Ashish Vaswani · Noam Shazeer · Niki Parmar · Jakob Uszkoreit · Llion Jones · Aidan N. Gomez · Lukasz Kaiser · Illia Polosukhin

Google Brain / Google Research

阿希什 · 瓦斯瓦尼 · 诺姆 · 沙泽尔 · 尼基 · 帕尔马 · 雅各布 · 乌什科赖特 · 利昂 · 琼斯 · 艾丹 · N · 戈麦斯 · 乌卡什 · 凯泽 · 伊利亚 · 波洛苏欣(均来自 Google)

Advances in Neural Information Processing Systems 30 (NIPS 2017), pp. 5998–6008 | arXiv:1706.03762 [cs.CL]

Abstract

摘要

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

当前主流的序列转换(sequence transduction)模型都基于复杂的循环神经网络或卷积神经网络,这些网络通常包含一个编码器和一个解码器。表现最好的模型还会通过注意力机制将编码器与解码器连接起来。我们提出了一种全新的、简洁的网络架构——Transformer,它完全基于注意力机制,完全摒弃了循环与卷积。在两个机器翻译任务上的实验表明,这些模型在质量上更优,同时具有更高的并行度,训练所需时间也显著减少。在 WMT 2014 英—德翻译任务上,我们的模型取得了 28.4 的 BLEU 分数,比当时已有的最佳结果(包括集成模型)还高 出 2 BLEU 以上。在 WMT 2014 英—法翻译任务上,我们的模型在 8 块 GPU 上仅训练了 3.5 天,便以单一模型取得了 41.8 的 BLEU 分数,刷新了该任务的最佳成绩,而训练成本 只是文献中最优模型的一小部分。我们还通过将 Transformer 成功应用于英文成分句法分 析(无论训练数据多寡)展示了它能够良好地泛化到其他任务。

1. Introduction

引言

Recurrent neural networks, long short-term memory and gated recurrent neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures.

循环神经网络,尤其是长短期记忆网络 (LSTM) 和门控循环单元 (GRU),已经成为序列建模与序列转换任务(如语言建模和机器翻译)中公认的最先进方法。此后,众多研究继续推动着循环语言模型和编码器—解码器架构的边界。

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t as a function of the previous hidden state h_{t−1} and the input for position t. This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks and conditional computation, while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains.

循环模型通常沿输入与输出序列的符号位置来分解计算。将这些位置与计算时间步对齐后,模型会根据前一隐藏状态 h_{t − 1} 与位置 t 处的输入,生成当前隐藏状态 h_t。这种内在的顺序性使得同一训练样本内部难以并行化,而随着序列变长,这会成为关键瓶颈——内存约束限制了跨样本的批大小。近期工作通过分解技巧与条件计算在计算效率上取得了显著改进,后者还同时提升了模型性能。然而,顺序计算这一根本性约束依然存 在。

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences. In all but a few cases, however, such attention mechanisms are used in conjunction with a recurrent network.

注意力机制已经成为许多引人注目的序列建模与转换模型的核心组成,使模型能够刻画依赖关系而不必在意其在输入或输出序列中的距离。然而,除少数情况外,这类注意力机制仍与循环网络结合使用。

In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs.

本文提出了一种名为 Transformer 的模型架构,它完全摒弃了循环结构,而是仅依靠注意力机制在输入与输出之间建立全局依赖。Transformer 显著提高了并行度,只需在 8 块 P100 GPU 上训练约 12 小时,即可在翻译质量上达到新的最优水平。

2. Background

背景

The goal of reducing sequential computation also forms the foundation of the Extended Neural GPU, ByteNet and ConvS2S, all of which use convolutional neural networks as basic building block, computing hidden representations in parallel for all input and output positions. In these models, the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes it more difficult to learn dependencies between distant positions. In the Transformer this is reduced to a constant number of operations, albeit at the cost of reduced effective resolution due to averaging attention-weighted positions, an effect we counteract with Multi-Head Attention as described in Section 3.2.

减少顺序计算这一目标也是 Extended Neural GPU、ByteNet 和 ConvS2S 这些工作的共同出发点:它们都使用卷积神经网络作为基本构建块,为所有输入和输出位置并行地计算隐藏表示。在这些模型中,关联任意两个输入或输出位置所需的运算量随位置之间的距离增长而增长——ConvS2S 中线性增长,ByteNet 中对数增长。这使得学习远距离依赖变得更 加困难。而在 Transformer 中,这一开销被降低为常数;代价是因为对注意力加权位置进行了平均,有效分辨率有所下降,我们将在 3.2 节中通过多头注意力机制抵消这一影响。

Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations.

自注意力(self-attention,有时也称为 intra-attention)是一种关联同一序列中不同位置的注意力机制,用于计算该序列的表示。自注意力已在多种任务中取得成功,包括阅读理解、抽象摘要、文本蕴含以及与具体任务无关的句子表示学习。

End-to-end memory networks are based on a recurrent attention mechanism instead of sequence-aligned recurrence and have been shown to perform well on simple-language question answering and language modeling tasks.

端到端记忆网络(end-to-end memory networks)基于循环注意力机制而非按序列对齐的循环结构,在简单的语言问答和语言建模任务上表现良好。

To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence-aligned RNNs or convolution. In the following sections, we will describe the Transformer, motivate self-attention and discuss its advantages over models such as neural GPUs, ByteNet and ConvS2S.

据我们所知,Transformer 是首个完全依赖自注意力来计算输入与输出表示的转换模型,它不使用任何按序列对齐的 RNN 或卷积。在接下来的章节中,我们将描述 Transformer 架构、阐述自注意力的动机,并讨论它相对于 Neural GPU、ByteNet 和 ConvS2S 等模型的 优势。

3. Model Architecture

模型架构

Most competitive neural sequence transduction models have an encoder-decoder structure. Here, the encoder maps an input sequence of symbol representations (x_1, ..., x_n) to a sequence of continuous representations z = (z_1, ..., z_n). Given z, the decoder then generates an output sequence (y_1,...,y_m) of symbols one element at a time. At each step the model is auto-regressive, consuming the previously generated symbols as additional input when generating the next.

大多数有竞争力的神经序列转换模型都采用编码器—解码器结构。其中,编码器将一个由符号表示组成的输入序列 (x_1, ..., x_n) 映射为连续表示序列 z = (z_1, ..., z_n); 给定 z,解码器再一个符号一个符号地生成输出序列 (y_1, ..., y_m)。在每一步,模型以自回归方式工作,将先前已生成的符号作为额外输入来生成下一个符号。

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

Transformer 沿用这种总体架构,在编码器和解码器中都使用堆叠的自注意力层和逐位(point-wise)全连接层,分别如图 1 的左半部分和右半部分所示。

3.1 Encoder and Decoder Stacks

编码器与解码器堆叠

Encoder: The encoder is composed of a stack of N = 6 identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection around each of the two sub-layers, followed by layer normalization. That is, the output of each sub-layer is LayerNorm(x + Sublayer(x)), where Sublayer(x) is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension d_model = 512.

编码器:编码器由 N = 6 个完全相同的层堆叠而成。每一层包含两个子层。第一个子层是多头自注意力机制,第二个子层是简单的逐位置(position-wise)全连接前馈网络。我们在两个子层周围都使用了残差连接(residual connection),再进行层归一化(layer normalization)。即每个子层的输出为 LayerNorm(x + Sublayer(x)),其中 Sublayer(x) 为该子层自身实现的变换。为便于这些残差连接,模型中所有子层以及嵌入层都产生维度 为 d_model = 512 的输出。

Decoder: The decoder is also composed of a stack of N = 6 identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i.

解码器:解码器同样由 N = 6 个完全相同的层堆叠而成。除了编码器层中的两个子层之外,解码器还插入了一个第三子层,该子层对编码器栈的输出执行多头注意力。与编码器类似,我们在每个子层周围使用残差连接,并进行层归一化。我们还修改了解码器栈中的自注意力子层,以防止位置关注到后续位置。这种掩码(masking)机制与输出嵌入偏移一个位置相结合,确保对位置 i 的预测只能依赖于位置小于 i 的已知输出。

3.2 Attention

注意力

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

注意力函数可以描述为:将一个查询(query)和一组键—值(key-value)对映射为一个输出,其中查询、键、值和输出都是向量。输出是各值向量的加权和,每个值向量所对应的权重由查询与相应键的兼容性函数(compatibility function)计算得到。

3.2.1 Scaled Dot-Product Attention

缩放点积注意力

We call our particular attention “Scaled Dot-Product Attention” (Figure 2). The input consists of queries and keys of dimension d_k, and values of dimension d_v. We compute the dot products of the query with all keys, divide each by √d_k, and apply a softmax function to obtain the weights on the values.

我们将本文所采用的注意力机制称为缩放点积注意力(Scaled Dot-Product Attention,图 2)。输入由维度为 d_k 的查询和键以及维度为 d_v 的值组成。我们计算查询与所有 键的点积,将每个点积除以 √ d_k,然后对其应用 softmax,得到对各值的权重。

Attention(Q, K, V) = softmax(Q K^T / √d_k) V

Attention(Q, K, V) = softmax(Q K^T / √ d_k) V

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q. The keys and values are also packed together into matrices K and V. We compute the matrix of outputs as the equation above.

实际实现时,我们会在一组查询上同时计算注意力函数,将这些查询打包成矩阵 Q;键和 值也分别打包成矩阵 K 和 V。我们按上式计算输出矩阵。

The two most commonly used attention functions are additive attention, and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of 1/√d_k. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

两种最常用的注意力函数分别是加性注意力(additive attention)和点积(乘性)注意力。点积注意力与本文算法完全一致,只是多了 1/ √ d_k 这一缩放因子。加性注意力使用一个单隐藏层的前馈网络来计算兼容性函数。两者在理论复杂度上相近,但实践中点积注意力速度更快、空间效率更高,因为它可以通过高度优化的矩阵乘法代码来实现。

While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k. We suspect that for large values of d_k, the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients. To counteract this effect, we scale the dot products by 1/√d_k.

当 d_k 较小时,两种机制表现相近;但 d_k 较大时,加性注意力优于未做缩放的点积注意力。我们推测:对于较大的 d_k,点积结果的数值会随之变大,从而将 softmax 推入其梯度极小的饱和区间。为抵消这一效应,我们将点积按 1/ √ d_k 进行缩放。

3.2.2 Multi-Head Attention

多头注意力

Instead of performing a single attention function with d_model-dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k, d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v-dimensional output values. These are concatenated and once again projected, resulting in the final values, as depicted in Figure 2.

我们发现,与使用 d_model 维的键、值和查询做一次注意力计算相比,将查询、键和值分 别通过 h 个不同的、学习到的线性投影映射到 d_k、d_k 和 d_v 维,然后在各自的投影版本上并行执行注意力函数,会更有帮助。每路输出为 d_v 维向量,这些向量被拼接起来再经过一次线性投影,得到最终输出,如图 2 所示。

MultiHead(Q, K, V) = Concat(head_1, ..., head_h) W^O; head_i = Attention(Q W^Q_i, K W^K_i, V W^V_i)

MultiHead(Q, K, V) = Concat(head_1, ..., head_h) W^O; head_i = Attention(Q W^Q_i, K W^K_i, V W^V_i)

Where the projections are parameter matrices W^Q_i ∈ ■^{d_model × d_k}, W^K_i ∈ ■^{d_model × d_k}, W^V_i ∈ ■^{d_model × d_v} and W^O ∈ ■^{h d_v × d_model}.

其中投影矩阵为参数矩阵 W^Q_i ∈ ^{d_model × d_k}、W^K_i ∈ ^{d_model × d_k}、W^V_i ∈ ^{d_model × d_v} 和 W^O ∈ ^{h d_v × d_model}。

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

多头注意力使模型能够同时关注来自不同表示子空间、不同位置的信息。而使用单头注意力时,平均操作会抑制这种能力。

In this work we employ h = 8 parallel attention layers, or heads. For each of these we use d_k = d_v = d_model / h = 64. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

本文中我们使用 h = 8 个并行的注意力层(即头)。每一头的维度为 d_k = d_v = d_model / h = 64。由于每个头的维度降低,多头注意力的总计算开销与单头全维度注意 力相近。

3.2.3 Applications of Attention in our Model

注意力在本文模型中的应用

The Transformer uses multi-head attention in three different ways:

Transformer 在三种不同场景下使用多头注意力:

• In “encoder-decoder attention” layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models.

• 在编码器—解码器注意力层中,查询来自上一层的解码器,而键和值来自编码器的输出。这使得解码器的每个位置都可以关注输入序列中的所有位置。这与典型序列到序列模型中的编码器—解码器注意力机制一致。

• The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.

• 编码器包含自注意力层。在自注意力层中,所有键、值和查询都来自同一来源——此处即编码器前一层的输出。编码器中的每个位置都可以关注前一层的所有位置。

• Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to −∞) all values in the input of the softmax which correspond to illegal connections.

• 同样地,解码器中的自注意力层允许解码器的每个位置关注解码器中截至并包括该位置的所有位置。我们需要阻止解码器中的信息向左流动,以保持自回归性质。具体做法是在缩放点积注意力内部,通过将 softmax 输入中所有非法连接对应的值置为 $-\infty$ 来屏 蔽掉这些位置。

3.3 Position-wise Feed-Forward Networks

逐位置前馈网络

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

除注意力子层外,编码器和解码器的每一层都包含一个全连接前馈网络,该网络对每个位置分别且相同地应用。它由两层线性变换和中间的一个 ReLU 激活函数组成。

$$FFN(x) = max(0, x W_1 + b_1) W_2 + b_2$$

$$\mathrm{FFN}(x) = \max(0, x W_1 + b_1) W_2 + b_2$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is d_model

= 512, and the inner-layer has dimensionality d_ff = 2048.

虽然不同位置上的线性变换是相同的,但层与层之间使用不同的参数。另一种等价的描述是将其视为两个卷积核大小为 1 的卷积。输入和输出的维度均为 d_model = 512,内层 维度为 d_ff = 2048。

3.4 Embeddings and Softmax

嵌入与 Softmax

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_model. We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation. In the embedding layers, we multiply those weights by √d_model.

与其他序列转换模型类似,我们使用学习到的嵌入将输入和输出符号转换为 d_model 维的向量。我们也使用常规的学习到的线性变换与 softmax 函数,将解码器的输出转换为下一个标记的预测概率。在本文模型中,两个嵌入层和 softmax 之前的线性变换共享同一权重矩阵。在嵌入层中,我们将这些权重乘以 √d_model。

3.5 Positional Encoding

位置编码

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add “positional encodings” to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_model as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed.

由于本文模型既无循环结构也无卷积结构,为使模型能利用序列的顺序信息,我们必须向序列中注入关于符号相对或绝对位置的信息。为此,我们在编码器和解码器栈底部的输入嵌入上叠加了位置编码。位置编码与嵌入具有相同的维度 d_model,因此二者可以直接相加。位置编码的选择既可以是学习得到的,也可以是固定的。

PE_{(pos, 2i)} = sin(pos / 10000^{2i / d_model})
PE_{(pos, 2i+1)} = cos(pos / 10000^{2i / d_model})

PE_{(pos, 2i)} = sin(pos / 10000^{2i / d_model})
PE_{(pos, 2i+1)} = cos(pos / 10000^{2i / d_model})

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to 10000 · 2π. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k, PE_{pos+k} can be represented as a linear function of PE_{pos}.

其中 pos 表示位置,i 表示维度。也就是说,位置编码的每个维度对应一个正弦波。这 些正弦波的波长构成从 2π 到 10000 · 2π 的等比数列。我们之所以选择这种函数,是因为我们假设它能让模型更容易学到按相对位置进行关注:对任意固定的偏移 k,PE_{pos+k} 都可以表示为 PE_{pos} 的线性函数。

We also experimented with using learned positional embeddings instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

我们也尝试了使用可学习的位置嵌入,发现两种版本的结果几乎相同(见表 3 第 (E) 行)。我们最终选择正弦版本,是因为它可能允许模型在训练时遇到的序列长度之外进行外推。

4. Why Self-Attention

为什么使用自注意力

In this section we compare various aspects of self-attention layers to the recurrent and convolutional layers commonly used for mapping one variable-length sequence of symbol representations (x_1, ..., x_n) to another sequence of equal length (z_1, ..., z_n), with x_i, z_i ∈ ■^d, such as a hidden layer in a typical sequence transduction encoder or decoder.

Motivating our use of self-attention we consider three desiderata.

本节中,我们将自注意力层与循环层、卷积层在多个方面进行比较——后者通常用于将一个变长的符号表示序列 (x_1, ..., x_n) 映射为另一个等长序列 (z_1, ..., z_n),其中 x_i, z_i ∈ ^d,典型应用即序列转换编码器/解码器中的一个隐藏层。为了说明我们选择自注意力的理由,我们考虑三个目标:

One is the total computational complexity per layer. Another is the amount of computation that can be parallelized, as measured by the minimum number of sequential operations required. The third is the path length between long-range dependencies in the network. Learning long-range dependencies is a key challenge in many sequence transduction tasks. One key factor affecting the ability to learn such dependencies is the length of the paths forward and backward signals have to traverse in the network. The shorter these paths between any combination of positions in the input and output sequences, the easier it is to learn long-range dependencies.

一是每一层的总计算复杂度;二是可被并行化的计算量(以所需的最少顺序操作数衡量);三是网络中长程依赖之间的路径长度。在许多序列转换任务中,学习长程依赖是一大挑战。影响学习这种依赖能力的一个关键因素是:在网络中,前向与后向信号必须穿越的路径长度。输入与输出序列中任意两个位置之间的路径越短,长程依赖就越容易学习。

Layer Type	Complexity per Layer	Sequential Ops	Max Path Length
Self-Attention	O(n² · d)	O(1)	O(1)
Recurrent	O(n · d²)	O(n)	O(n)
Convolutional	O(k · n · d²)	O(1)	O(log_k(n))
Self-Attention (restricted)	O(r · n · d)	O(1)	O(n/r)

Table 1 — Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the neighborhood size in restricted self-attention.

■ *1* —— ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ *n* ■■■■■ ■,*d* ■■■■■■,*k* ■■■■■■■,*r* ■■■■■■■■■■■■■■

As noted in Table 1, a self-attention layer connects all positions with a constant number of sequentially executed operations, whereas a recurrent layer requires O(n) sequential operations. In terms of computational complexity, self-attention layers are faster than recurrent layers when the sequence length n is smaller than the representation dimensionality d, which is most often the case with sentence representations used by state-of-the-art models in machine translations, such as word-piece and byte-pair representations. To improve computational performance for tasks involving very long sequences, self-attention could be restricted to considering only a neighborhood of size r in the input sequence centered around the respective output position. This would increase the maximum path length to O(n/r). We plan to investigate this approach further in future work.

如表 1 所示,自注意力层以常数个顺序操作连接所有位置,而循环层则需要 O(n) 个顺序操作。在计算复杂度方面,当序列长度 n 小于表示维度 d 时,自注意力层快于循环层——这在机器翻译的最先进模型(如基于 word-piece 或 byte-pair 的表示)所处理的句子表示中往往是成立的。为了在涉及极长序列的任务中提升计算效率,自注意力可以被限制为:仅考虑输入序列中以相应输出位置为中心、大小为 r 的邻域。这种做法将最大路径 长度增加到 O(n/r)。我们计划在未来工作中进一步研究这一方法。

A single convolutional layer with kernel width k < n does not connect all pairs of input and output positions. Doing so requires a stack of O(n/k) convolutional layers in the case of contiguous kernels, or O(log_k(n)) in the case of dilated convolutions, increasing the length of the longest paths between any two positions in the network. Convolutional layers are generally more expensive than recurrent layers, by a factor of k. Separable convolutions, however, decrease the complexity considerably, to O(k · n · d + n · d²). Even with k = n, however, the complexity of a separable convolution is equal to the combination of a self-attention layer and a point-wise feed-forward layer, the approach we take in our model.

对于卷积核宽度 k < n 的单层卷积,它无法连接输入与输出之间的所有位置对。要做到这一点,在使用连续卷积核的情况下需要堆叠 O(n/k) 个卷积层,在使用扩张卷积的情况下 需要 O(log_k(n)) 层,这会使网络中任意两个位置之间的最长路径变长。卷积层一般比 循环层贵 k 倍。然而可分离卷积(separable convolution)可以将复杂度大幅降低到 O(k · n · d + n · d²)。即便 k = n,可分离卷积的复杂度也等于一个自注意力层加

一个逐位置前馈层的组合——这正是本文模型所采用的方案。

As side benefit, self-attention could yield more interpretable models. We inspect attention distributions from our models and present and discuss examples in the appendix. Not only do individual attention heads clearly learn to perform different tasks, many appear to exhibit behavior related to the syntactic and semantic structure of the sentences.

附带的好处是,自注意力有助于产生更具可解释性的模型。我们检查了模型中的注意力分布,并在附录中展示与讨论了相关示例。不仅是各个注意力头明显地学会了完成不同的任务,其中许多还表现出与句子句法结构、语义结构相关的行为。

5. Training

训练

5.1 Training Data and Batching

训练数据与批处理

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. Sentences were encoded using byte-pair encoding, which has a shared source-target vocabulary of about 37000 tokens. For English-French, we used the significantly larger WMT 2014 English-French dataset consisting of 36M sentences and split tokens into a 32000 word-piece vocabulary. Sentence pairs were batched together by approximate sequence length. Each training batch contained a set of sentence pairs containing approximately 25000 source tokens and 25000 target tokens.

我们在标准的 WMT 2014 英—德数据集上进行训练,数据集包含约 450 万对句子。句子采用字节对编码(byte-pair encoding),源端与目标端共享一个约 37 000 个标记的词表。对于英—法任务,我们使用了规模更大的 WMT 2014 英—法数据集,包含 3600 万句, 并将标记切分为 32 000 个 word-piece 的词表。我们按大致相近的序列长度对句对进行 分批;每个训练批包含约 25 000 个源端标记和 25 000 个目标端标记。

5.2 Hardware and Schedule

硬件与训练计划

We trained our models on one machine with 8 NVIDIA P100 GPUs. For our base models using the hyperparameters described throughout the paper, each training step took about 0.4 seconds. We trained the base models for a total of 100,000 steps or 12 hours. For our big models (described on the bottom line of Table 3), step time was 1.0 seconds. The big models were trained for 300,000 steps (3.5 days).

我们在配备 8 块 NVIDIA P100 GPU 的单台机器上训练模型。对于使用本文中所述超参数的基础模型,每一步训练耗时约 0.4 秒;基础模型总共训练了 100 000 步,合计 12 小时。 对于大型模型(对应表 3 最后一行),每步耗时 1.0 秒,训练 300 000 步,合计 3.5 天。

5.3 Optimizer

优化器

We used the Adam optimizer with β_1 = 0.9, β_2 = 0.98 and ε = 10^–9. We varied the learning rate over the course of training, according to the formula:

我们使用 Adam 优化器,其中 β_1 = 0.9、 β_2 = 0.98、 ε = 10^ − 9。训练过程中,我们按如下公式调整学习率:

$$lrate = d\_model^{-0.5} \cdot min(step\_num^{-0.5}, step\_num \cdot warmup\_steps^{-1.5})$$

lrate = d_model^{ − 0.5} · min(step_num^{ − 0.5}, step_num · warmup_steps^{ − 1.5})

This corresponds to increasing the learning rate linearly for the first warmup_steps training steps, and decreasing it thereafter proportionally to the inverse square root of the step number. We used warmup_steps = 4000.

这等价于:在前 warmup_steps 步中线性增大学习率,之后按步数的反平方根成比例减小。 我们设置 warmup_steps = 4000。

5.4 Regularization

正则化

We employ three types of regularization during training:

我们在训练中使用了三种正则化方法:

Residual Dropout: We apply dropout to the output of each sub-layer, before it is added to the sub-layer input and normalized. In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of P_drop = 0.1.

残差 Dropout:我们对每个子层的输出应用 dropout,然后再将其与子层输入相加并归一化。此外,在编码器和解码器栈中,我们对嵌入与位置编码之和也应用 dropout。对于基础模型, 我们使用 P_drop = 0.1 的丢弃率。

Label Smoothing: During training, we employed label smoothing of value ε_ls = 0.1. This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score.

标签平滑(Label Smoothing):训练中我们采用 ε _ls = 0.1 的标签平滑。虽然这会使模型更加不确定,从而导致困惑度上升,但能提升准确率与 BLEU 分数。

6. Results

实验结果

6.1 Machine Translation

机器翻译

On the WMT 2014 English-to-German translation task, the big transformer model (Transformer (big) in Table 2) outperforms the best previously reported models (including ensembles) by more than 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. The configuration of this model is listed in the bottom line of Table 3. Training took 3.5 days on 8 P100 GPUs. Even our base model surpasses all previously published models and ensembles, at a fraction of the training cost of any of the competitive models.

在 WMT 2014 英—德翻译任务上,大型 Transformer 模型(表 2 中的 Transformer (big))以 28.4 的 BLEU 分数刷新了当时的最佳成绩,比此前报道的最佳模型(包括集成模型)高出 2.0 BLEU 以上。该模型的配置列于表 3 的最后一行,在 8 块 P100 GPU 上训练了 3.5 天。甚至我们的基础模型也以远低于所有竞品模型的训练成本,超越了此前所有已发 表的模型与集成。

On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.0, outperforming all of the previously published single models, at less than 1/4 the training cost of the previous state-of-the-art model. The Transformer (big) model trained for English-to-French used dropout rate P_drop = 0.1, instead of 0.3.

在 WMT 2014 英—法翻译任务上,我们的大型模型取得了 41.0 的 BLEU 分数,超过了此前所有已发表的单模型成绩,而其训练成本不到此前最佳模型的 1/4。在英—法任务上, Transformer (big) 模型使用的 dropout 率为 P_drop = 0.1(而不是 0.3)。

For the base models, we used a single model obtained by averaging the last 5 checkpoints, which were written at 10-minute intervals. For the big models, we averaged the last 20 checkpoints. We used beam search with a beam size of 4 and length penalty α = 0.6. These hyperparameters were chosen after experimentation on the development set. We set the maximum output length during inference to input length + 50, but terminate early when possible.

对于基础模型,我们将最后 5 个检查点(每 10 分钟保存一次)进行平均,作为最终模型。对于大型模型,我们平均最后 20 个检查点。我们使用束大小为 4、长度惩罚 α = 0.6 的束搜索。这些超参数是在开发集上实验得到的。推理时,我们将最大输出长度设为输入长 度 + 50,并在可能时提前终止生成。

Model	EN-DE BLEU	EN-FR BLEU		EN-DE FLOPs	EN-FR FLOPs
ByteNet	23.75	—		—	—
Deep-Att + PosUnk	—	39.2		—	$1.0 \cdot 10^{2}$■
GNMT + RL	24.6	39.92		$2.3 \cdot 10^{1}$■	$1.4 \cdot 10^{2}$■
ConvS2S	25.16	40.46		$9.6 \cdot 10^{1}$■	$1.5 \cdot 10^{2}$■
MoE	26.03	40.56		$2.0 \cdot 10^{1}$■	$1.2 \cdot 10^{2}$■
Deep-Att + PosUnk (Ens.)	—	40.4		—	$8.0 \cdot 10^{2}$■

GNMT + RL (Ens.)	26.30	41.16		$1.8 \cdot 10^{2}$■	$1.1 \cdot 10^{21}$
ConvS2S (Ens.)	26.36	41.29		$7.7 \cdot 10^{1}$■	$1.2 \cdot 10^{21}$
Transformer (base)	27.3	38.1		$3.3 \cdot 10^{1}$■	
Transformer (big)	28.4	41.8		$2.3 \cdot 10^{1}$■	

Table 2 — The Transformer achieves better BLEU scores than previous state-of-the-art models on the WMT 2014 English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

■ 2 —— ■ *WMT 2014* ■—■■■—■ *newstest2014* ■■■■,*Transformer* ■■■■ ■■■■■■■■■■■■■■■■ *BLEU* ■■■

6.2 Model Variations

模型变体

To evaluate the importance of different components of the Transformer, we varied our base model in different ways, measuring the change in performance on English-to-German translation on the development set, newstest2013. We used beam search as described in the previous section, but no checkpoint averaging. We present these results in Table 3.

为了评估 Transformer 不同组件的重要性,我们以多种方式改变基础模型,并在英—德翻 译的 dev 集 newstest2013 上测量性能变化。我们使用与上文相同的束搜索,但不做检 查点平均。实验结果列于表 3。

In Table 3 rows (A), we vary the number of attention heads and the attention key and value dimensions, keeping the amount of computation constant, as described in Section 3.2.2. While single-head attention is 0.9 BLEU worse than the best setting, quality also drops off with too many heads.

在表 3 的 (A) 行中,我们改变注意力头的数量以及注意力键和值的维度,同时保持计算 量不变(如 3.2.2 节所述)。单头注意力比最佳设置差 0.9 BLEU;而当头的数量过多时, 质量也会下降。

In Table 3 rows (B), we observe that reducing the attention key size d_k hurts model quality. This suggests that determining compatibility is not easy and that a more sophisticated compatibility function than dot product may be beneficial. We further observe in rows (C) and (D) that, as expected, bigger models are better, and dropout is very helpful in avoiding over-fitting. In row (E) we replace our sinusoidal positional encoding with learned positional embeddings, and observe nearly identical results to the base model.

在表 3 的 (B) 行中,我们发现减小注意力键的维度 d_k 会损害模型质量。这表明,相容 性函数的判断并非易事,采用比点积更复杂的相容性函数可能带来好处。我们进一步在 (C) 和 (D) 行中观察到:正如预期,更大的模型效果更好;dropout 对避免过拟合非常有 帮助。在 (E) 行中,我们用可学习的位置嵌入替换正弦位置编码,得到了与基础模型几 乎相同的结果。

6.3 English Constituency Parsing

英文成分句法分析

To evaluate if the Transformer can generalize to other tasks we performed experiments on English constituency parsing. This task presents specific challenges: the output is subject to strong structural constraints and is significantly longer than the input. Furthermore, RNN sequence-to-sequence models have not been able to attain state-of-the-art results in small-data regimes.

为了检验 Transformer 能否泛化到其他任务,我们在英文成分句法分析任务上进行了实 验。该任务带来了一些特殊的挑战:输出受到很强的结构约束,且输出长度显著大于输入 长度。此外,在小数据场景下,RNN 序列到序列模型始终无法取得最优结果。

We trained a 4-layer transformer with d_model = 1024 on the Wall Street Journal (WSJ) portion of the Penn Treebank, about 40K training sentences. We also trained it in a semi-supervised setting, using the larger high-confidence and BerkleyParser corpora from with approximately 17M sentences. We used a vocabulary of 16K tokens for the WSJ only setting and a vocabulary of 32K tokens for the semi-supervised setting.

我们在 Penn Treebank 的华尔街日报 (WSJ) 部分(约 4 万训练句)上训练了一个 d_model = 1024 的 4 层 Transformer。我们也在半监督设置下进行了训练,使用了更大规模的 high-confidence 与 BerkleyParser 语料(约 1700 万句)。WSJ-only 设置下使用 16 000 个标记的词表,半监督设置下使用 32 000 个标记的词表。

We performed only a small number of experiments to select the dropout, both attention and residual, learning rates and beam size on the Section 22 development set, all other parameters remained unchanged from the English-to-German base translation model. During inference, we increased the maximum output length to input length + 300. We used a beam size

of 21 and α = 0.3 for both WSJ only and the semi-supervised setting.

我们仅在 Section 22 开发集上做了少量实验,用以选择 dropout(注意力 dropout 与残差 dropout)、学习率以及束大小;其余参数保持与英—德翻译基础模型相同。推理时,我们将最大输出长度增加到 input length + 300。WSJ-only 与半监督两种设置下,我们都使 用束大小 21 与 α = 0.3。

Our results in Table 4 show that despite the lack of task-specific tuning our model performs surprisingly well, yielding better results than all previously reported models with the exception of the Recurrent Neural Network Grammar. In contrast to RNN sequence-to-sequence models, the Transformer outperforms the BerkeleyParser even when training only on the WSJ training set of 40K sentences.

表 4 的结果表明:尽管缺乏针对特定任务的调优,本文模型表现惊人地好,在除 Recurrent Neural Network Grammar 之外的所有此前报道的模型中取得了最好的结果。与基于 RNN 的序列到序列模型不同,即便仅使用 40 000 句的 WSJ 训练集,Transformer 也超过了 BerkeleyParser。

Parser	Training	WSJ 23 F1
Vinyals & Kaiser et al. (2014)	WSJ only, discriminative	88.3
Petrov et al. (2006)	WSJ only, discriminative	90.4
Zhu et al. (2013)	WSJ only, discriminative	90.4
Dyer et al. (2016)	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013)	Semi-supervised	91.3
Huang & Harper (2009)	Semi-supervised	91.3
McClosky et al. (2006)	Semi-supervised	92.1
Vinyals & Kaiser et al. (2014)	Semi-supervised	92.1
Transformer (4 layers)	Semi-supervised	92.7
Luong et al. (2015)	Multi-task	93.0
Dyer et al. (2016)	Generative	93.3

Table 4 — The Transformer generalizes well to English constituency parsing (Results are on Section 23 of WSJ).

■ 4 —— Transformer ■■■■■■■■■■■■■■■■■■■■■■(■■■ WSJ ■ Section 23 ■■■■)■

7. Conclusion

结论

In this work, we presented the Transformer, the first sequence transduction model based entirely on attention, replacing the recurrent layers most commonly used in encoder-decoder architectures with multi-headed self-attention.

本文中,我们提出了 Transformer——第一个完全基于注意力的序列转换模型。它在编码器—解码器架构中,用多头自注意力替代了最常用的循环层。

For translation tasks, the Transformer can be trained significantly faster than architectures based on recurrent or convolutional layers. On both WMT 2014 English-to-German and WMT 2014 English-to-French translation tasks, we achieve a new state of the art. In the former task our best model outperforms even all previously reported ensembles.

在翻译任务上,Transformer 的训练速度明显快于基于循环层或卷积层的架构。在 WMT 2014 英—德与英—法两个翻译任务上,我们都达到了新的最佳水平;在前一个任务上,我们的最佳模型甚至超过了所有此前报道的集成模型。

We are excited about the future of attention-based models and plan to apply them to other tasks. We plan to extend the Transformer to problems involving input and output modalities other than text and to investigate local, restricted attention mechanisms to efficiently handle large inputs and outputs such as images, audio and video. Making generation less sequential is another research goals of ours.

我们对基于注意力的模型的未来充满期待,并计划将其应用于其他任务。我们计划将 Transformer 扩展到文本以外的输入输出模态(如图像、音频和视频)的问题中,并研究局部化、受限的注意力机制,以高效处理超长输入输出。降低生成过程的顺序性也是我们的 研究方向之一。

The code we used to train and evaluate our models is available at https://github.com/tensorflow/tensor2tensor.

我们用于训练与评估模型的代码已开源,地址:https://github.com/tensorflow/tensor2tensor。

Acknowledgements

致谢

We are grateful to Nal Kalchbrenner and Stephan Gouws for their fruitful comments, corrections and inspiration.

我们感谢 Nal Kalchbrenner 和 Stephan Gouws 给予的富有成效的评论、纠正与启发。

Note: This is a faithful, full-text translation of the paper as published in NeurIPS 2017. Figures have been referenced by number but not re-rendered; please consult the original paper for figures. All mathematical formulas are reproduced in linear notation (e.g., QK^T / √d_k).